Policy Assistant Agent — Analysis

Week 27 Graded Mini Project · Samson Elias · Single agent + memory + graph orchestration · gpt-4o-mini (temperature 0, seed 42) via Vocareum · LangChain, LangGraph, FAISS (CPU)

1. Summary

I built a policy question-answering agent that answers only from a local corpus of 10 internal policies. It uses three tools (FAISS policy_search, read_policy, safe calculator), cites a **[doc_id]** and a verbatim quote in every answer, recovers from a bad doc_id, stores only verified facts in a FAISS memory, and runs inside a LangGraph with a step budget, a critic and one retry. Every acceptance criterion was met: final answers are grounded and cited, runs finish within max_iterations (6) and max_steps (8), memory is written only for verified facts, and **8/8 queries (100%)** pass in the final evaluation (target ≥ 80%).

2. Design

- **Tools with typed errors:** read_policy raises PolicyNotFoundError (code POLICY_NOT_FOUND, plus a hint); the LangChain wrapper turns it into a soft JSON error, so the agent can recover instead of crashing.
- **Prompt acceptance rules:** R1 grounding, R2 [doc_id] token, R3 verbatim quote (≤ 30 words), R4 step limits, R5 bad-id recovery, R6 memory hint = lead to re-verify. A fixed Answer/Quote format makes the rules checkable by code.
- **Graph:** plan → memory_read → agent_answer → critic → memory_write → stop. Each node counts a step; routers stop at max_steps (budget_exhausted); a critic failure triggers one retry with the issues as feedback.
- **Memory:** write only if critic valid + real doc_id + verbatim quote. Read cheapest-sufficient first: exact match, then FAISS cosine ≥ 0.80 (shortest qualifying entry), else use the tools.
- **Critic:** rules C1 doc_id present, C2 cited doc exists and was read, C3 verbatim quote, C4 numbers grounded, plus an LLM critic in JSON mode. The answer is valid only if both pass.

3. Results

Run	Pass rate	Avg latency	Avg tool calls	Memory hits	Notes
Eval v1 — agent only	8/8 (100%)	5.6 s	2.25	–	recovery 3 calls, multi-tool 3 calls
Eval v2 — graph, cold	8/8 (100%)	8.1 s	2.25	0/8	all 8 facts verified and written
Eval v2 — warm (before fix)	7/8 (88%)	4.9 s	1.25	8/8	Q6 omitted the calculation; Q2 retried once
Eval v2 — warm (after R6 fix)	8/8 (100%)	5.2 s	1.50	8/8	Q6 calls calculator again

Demo runs: cold 7.2 s / 2 tool calls; warm (memory hit) 3.7 s / 1 tool call; error recovery read_policy ERROR → policy_search → read_policy ok, citing [remote_work_v3]. **Retrieval:** the correct policy ranked first for every starter query with a clear margin (e.g. 0.61 vs 0.34). **Tests:** 34 pytest tests pass (tools, acceptance, budget stop, retry, memory, SQLite checkpointing).

4. Key findings

- **Memory pays off.** Warm runs cut tool calls by about a third (2.25 → 1.5) and latency by about 36% (8.1 → 5.2 s), because the hint lets the agent skip policy_search and go straight to read_policy.
- **A memory hint can cause partial answers.** On the warm run, Q6 ("Compute 12 * 4 and then tell me the meals cap") answered only the meals cap: the hint focused the model on the stored fact. One sentence added to R6 ("the hint covers only the policy fact — still answer every part of the question") fixed it: 7/8 → 8/8.
- **The critic checks what is present, not what is missing.** The incomplete Q6 answer passed the critic (no bad citation, quote or number); the eval's keyword check caught it. Similarly, the agent recovered from the bad id but did not tell the user the id was missing (R5), and no check flags that.
- **Two critics are better than one.** For a wrong "4 days" answer, rule C4 flagged the number and the LLM critic explained the contradiction with the policy ("up to 3 days per week"). The retry path also fired on a real case (Q2 warm), and the graph's budget stop and single retry behaved exactly as designed.
- **The threshold trades recall for safety.** A close paraphrase scored 0.80 (hit), while "How many days can I work from home?" missed. A missed hint costs one extra tool call, while a wrong hint could mislead, so a strict threshold is the safer default.

5. Limitations and next steps

- Add a **completeness check** to the critic (does the answer address every part of the question?) and an R5 check (when recovery happened, does the answer say the requested id was not found?).
- C4 checks numbers only; claims without numbers are checked only by the LLM critic. Memory has no expiry, so facts must be cleared when a policy changes; facts could be tagged with the policy date and invalidated

automatically.

- OpenAI's seed gives best-effort determinism, so the unit tests use a deterministic offline model, and live results can vary slightly between runs. The corpus is small (10 policies); a larger corpus would need chunking and a re-ranking step.

6. Acceptance criteria — evidence

Criterion	Evidence	Met
Grounded answers only; no external facts	Prompt R1; critic C2 (cited doc read this run) and C3 (verbatim quote); LLM critic checks claims against evidence	Yes
Final answers include doc_id	has_docid = True on 8/8 (v1, v2 cold, v2 warm); critic C1	Yes
Short quotes for exact rules	has_quote = True on 8/8; every quote verified verbatim against the cited policy	Yes
Within max_iterations and graph max_steps	Agent max 4 of 6 iterations; graph 5 of 8 steps (7 with retry); budget stop verified at max_steps = 3	Yes
Memory writes only for verified facts	Rejected: no doc_id, fake quote, unknown id, critic-invalid; nothing written on critic_failed or budget stop	Yes
≥ 80% queries pass acceptance checks	v1 100%, v2 cold 100%, v2 warm 88% → 100% after R6 fix	Yes
Outputs: eval table, logs, cold / warm / recovery	eval.py tables; logs/*.log + *.jsonl + results/*.csv per run; app.py demo + screenshots	Yes
Extras	SQLite checkpointing (--sqlite); JSON run report (--report); per-node error wrapper; LLM critic	Yes